UNIVERSIDAD MADERO

LICENCIATURA EN INGENIERÍA EN SEGURIDAD INFORMÁTICA Y REDES

Matricula:

20145025

Semestre:

3er. Semestre

Alumno:

Gabriel René Díaz Lara

Materia:

Delitos Cibernéticos

Docente:

Dulce María Vázquez Caro

Fecha:

Septiembre 2026

Contenido

1. **Introducción** 3
2. **Desarrollo del Laboratorio.** 4
   - **2.1 Práctica 1. Reconocimiento de red y escaneo de puertos** 5
   - **2.2 Práctica 2. Captura y análisis de tráfico ARP, DNS y TCP** 9
   - **2.3 Práctica 3. Criptografía en acción: hash, AES, RSA y TLS** 15
3. **Respuestas de estudiante** 23
4. **Conclusión** 26
5. **Bibliografía** 27

Actividad 2: LabSemana2 Conceptos de Seguridad en Redes.

1. Introducción

La seguridad perimetral clásica lleva años demostrando fisuras, pero ver caer esos supuestos en un entorno local siempre deja una sensación distinta. Muchos administradores asumen que, si un equipo ya está dentro del segmento LAN, el entorno es intrínsecamente seguro; la realidad del tráfico diario desmiente esa premisa en cuestión de minutos. Los protocolos base sobre los que todavía corre buena parte de la comunicación corporativa se diseñaron en una época donde la interoperabilidad importaba más que la autenticación, dejando puertas abiertas que hoy cualquier atacante novato sabe explotar con herramientas públicas.

¿Tiene sentido seguir confiando a ciegas en tramas locales cuando cualquiera con acceso al medio físico puede redirigir flujos enteros? Difícilmente.

En este laboratorio se montó un escenario controlado de dos máquinas virtuales —Kali Linux como estación de análisis y Ubuntu Server como nodo objetivo— para poner a prueba de forma directa tres vectores elementales: el reconocimiento de servicios, la manipulación de tráfico en capas de enlace y transporte, y la aplicación real de primitivas criptográficas. No se trata solo de tirar comandos de Nmap o abrir Wireshark por cumplir con un temario; el valor está en inspeccionar byte por byte cómo viajan las cabeceras, por qué un switch traga anuncios ARP falsos sin chistar y qué ocurre exactamente cuando un payload pasa de texto plano a un bloque cifrado bajo estándares modernos.

La seguridad en redes no puede concebirse como un elemento añadido a posteriori, sino como un conjunto de servicios de autenticación, control de acceso y confidencialidad integrados desde la arquitectura base. Cuando esa integración falla o se delega a la buena fe de los protocolos heredados, el aislamiento lógico de una subred desaparece por completo (algo que queda patente al revisar las capturas de ARP y DNS obtenidas durante la sesión).

A lo largo de las siguientes secciones se desglosan los hallazgos técnicos de cada práctica, contrastando la visibilidad del tráfico no protegido frente a las garantías de integridad y confidencialidad que ofrecen algoritmos como AES-256, RSA y el

protocolo TLS 1.3, integrando además una lectura crítica sobre las implicaciones operativas y legales que estos vectores conllevan en el contexto normativo mexicano.

2. Desarrollo del Laboratorio.

Montar un laboratorio de este tipo sobre un solo hipervisor exige tomar decisiones prácticas desde el minuto cero. Para la ejecución de las tres prácticas se implementó una topología punto a punto virtualizada sobre VMware Workstation, compuesta por dos instancias: una máquina atacante/analizadora corriendo **Kali Linux** (con el arsenal clásico de Nmap, OpenSSL, Wireshark y suites de manipulación de tramas) y un servidor objetivo corriendo **Ubuntu Server** con servicios base instalados manualmente para emular un host corporativo típico (específicamente OpenSSH, ISC BIND y Apache httpd).

La configuración original de los adaptadores de red, sin embargo, nos jugó una pequeña mala pasada durante la sesión que vale la pena documentar con total transparencia.

Inicialmente, ambas máquinas se levantaron en un conmutador virtual aislado tipo *Host-Only* bajo el segmento 192.168.234.0/24. La lógica detrás de esa primera elección era meramente precautoria: contener el ruido de los escaneos de puertos y la inyección de tramas dentro de un espacio estanco donde ninguna ráfaga de paquetes pudiera salir por error hacia la interfaz física de la máquina host. Bajo este direccionamiento inicial —con Kali posicionado en 192.168.234.128 y Ubuntu en 192.168.234.129— se desarrollaron de punta a punta la **Práctica 1** (reconocimiento y escaneo de puertos) y la primera mitad de la **Práctica 2** (análisis del intercambio ARP y emisión de tramas gratuitas con arping).

El problema saltó al llegar a las pruebas de resolución de nombres DNS públicos y la verificación de handshakes TLS contra servidores externos. Al ser un segmento estrictamente local sin puerta de enlace predeterminada hacia el exterior, cualquier intento de lanzar un dig google.com o conectar a un host seguro con curl arrojaba de inmediato un error de red inalcanzable (Network is unreachable). Mantener el entorno 100% aislado ya no era viable si queríamos capturar tráfico real de Internet y contrastarlo con el tráfico local.

Se tomó entonces la decisión operativa de migrar las interfaces virtuales de ambas máquinas hacia un esquema de puente (*Bridged Adapter*). Esto provocó que el servidor DHCP del router físico de la red local les reasignara nuevas direcciones dentro del segmento 192.168.1.0/24:

- **VM1 (Atacante/Analista):** Kali Linux — 192.168.1.102 (MAC física virtual: 00:0c:29:0a:21:0a)
- **VM2 (Víctima/Servidor):** Ubuntu Server — 192.168.1.101 (MAC física virtual: 00:0c:29:e8:b3:fc)
- **Puerta de enlace / Servidor DNS local:** 192.168.1.254

Esta transición de direccionamiento explica de forma directa por qué en las evidencias de la primera fase aparecen IPs del rango .234.x y, a partir del análisis del handshake TCP y las pruebas de TLS, las capturas reflejan el segmento .1.x. Lejos de restar validez a la práctica, este ajuste sobre la marcha permitió mantener la visibilidad bidireccional entre ambos nodos sin sacrificar la salida a la WAN necesaria para las pruebas criptográficas de la última etapa.

2.1 Práctica 1. Reconocimiento de red y escaneo de puertos

La primera práctica tuvo como propósito identificar los equipos activos dentro del segmento de red y reconocer los servicios expuestos por Ubuntu. El análisis se realizó desde Kali Linux mediante Nmap y Wireshark. Inicialmente, las máquinas se encontraban en el segmento aislado 192.168.234.0/24, por lo que las primeras evidencias corresponden a ese direccionamiento.

- Descubrimiento de hosts

Se ejecutó el comando:

nmap -sn 192.168.234.0/24

El resultado indicó que se encontraron cuatro hosts activos dentro del segmento analizado:

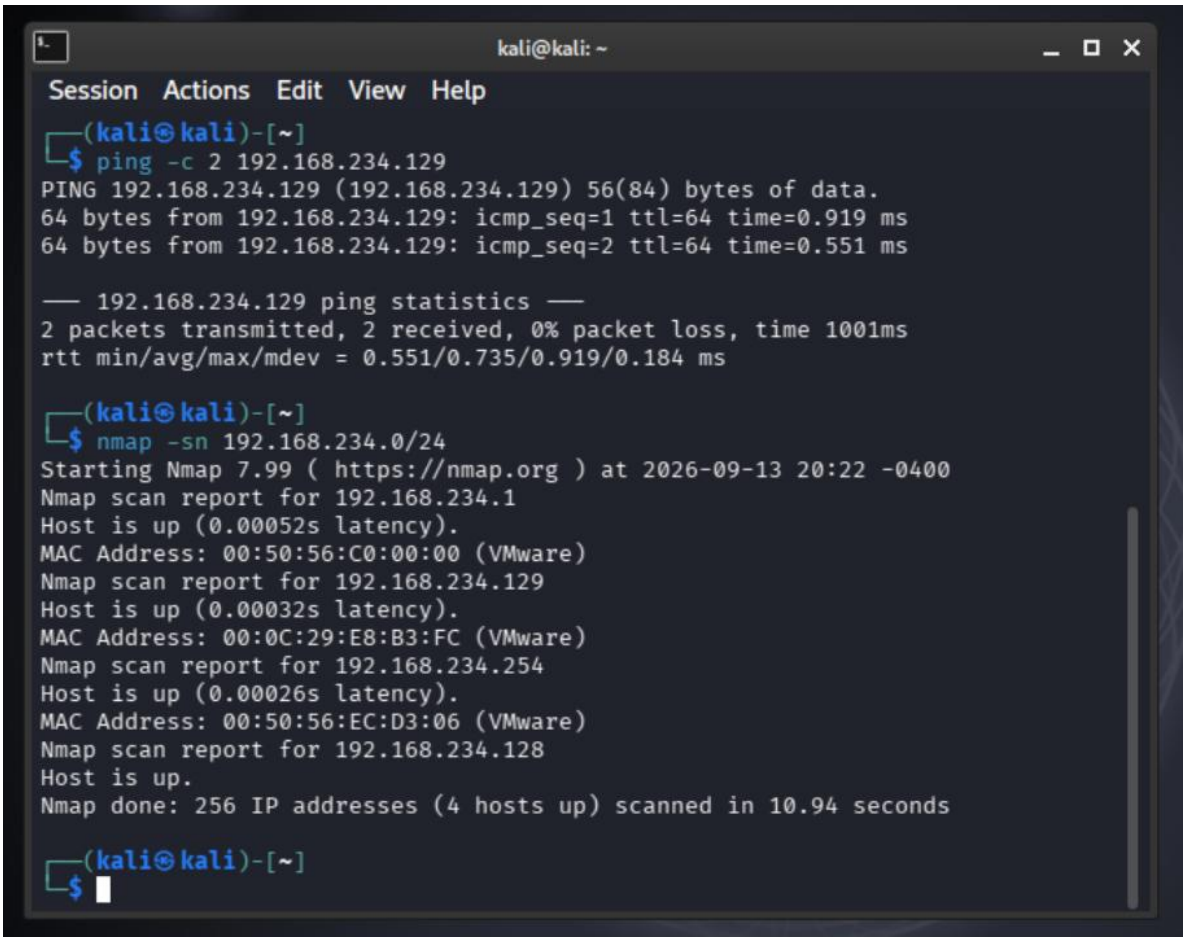
- 192.168.234.1
- 192.168.234.128, correspondiente a Kali Linux.
- 192.168.234.129, correspondiente a Ubuntu.
- 192.168.234.254

La dirección MAC identificada para Ubuntu fue:

00:0c:29:e8:b3:fc

La evidencia también muestra que el escaneo recorrió las 256 direcciones del segmento y encontró cuatro hosts activos.

Figura 1. Descubrimiento de hosts activos en el segmento local mediante barrido ICMP/ARP.



Ejecución de nmap -sn sobre el segmento de red local desde VM1 (Kali Linux). Se evidencian las direcciones IP de los hosts activos en la subred, confirmando la presencia y alcance de la máquina objetivo VM2 (Ubuntu).

- **Enumeración de puertos y servicios**

Posteriormente, se ejecutó un escaneo de detección de servicios y versiones contra Ubuntu:

nmap -sV 192.168.234.129

El resultado mostró los siguientes puertos abiertos:

Puerto	Estado	Servicio	Versión detectada
21/tcp	Open	FTP	vsftpd 3.0.5
22/tcp	Open	SSH	OpenSSH 10.0p2 Ubuntu 5ubuntu5.1

Puerto	Estado	Servicio	Versión detectada
53/tcp	Open	Domain	ISC BIND 9.20.11-1ubuntu2.2
80/tcp	Open	HTTP	Apache httpd 2.4.64
443/tcp	Open	SSL/HTTP	Apache httpd 2.4.64

La información obtenida permite identificar la superficie de exposición de la máquina. SSH podría utilizarse para administración remota, Apache presta el servicio web y BIND participa en la resolución de nombres. FTP debe considerarse especialmente delicado si permite autenticación o transferencia sin cifrado. También es importante recordar que la detección de un puerto abierto no demuestra por sí sola que exista una vulnerabilidad; indica que hay un servicio accesible que debe mantenerse actualizado y correctamente configurado.

Figura 2. Enumeración de puertos y detección de versiones de servicios en la máquina objetivo.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.234.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-13 20:25 -0400
Nmap scan report for 192.168.234.129
Host is up (0.00033s latency).
Not shown: 995 closed tcp ports (reset)
PORT    STATE SERVICE  VERSION
21/tcp  open  ftp      vsftpd 3.0.5
22/tcp  open  ssh      OpenSSH 10.0p2 Ubuntu 5ubuntu5.1 (Ubuntu Linux; proto
col 2.0)
53/tcp  open  domain   ISC BIND 9.20.11-1ubuntu2.2 (Ubuntu Linux)
80/tcp  open  http     Apache httpd 2.4.64
443/tcp open  ssl/http Apache httpd 2.4.64
MAC Address: 00:0C:29:E8:B3:FC (VMware)
Service Info: Host: 127.0.1.1; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_ker
nel

Service detection performed. Please report any incorrect results at https://
nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 28.23 seconds
```

Escaneo de servicios y versiones con nmap -sV hacia VM2 (Ubuntu). Se identifican los puertos abiertos junto con las versiones exactas de OpenSSH y Apache, permitiendo caracterizar la superficie de exposición del sistema.

- **Captura de paquetes SYN**

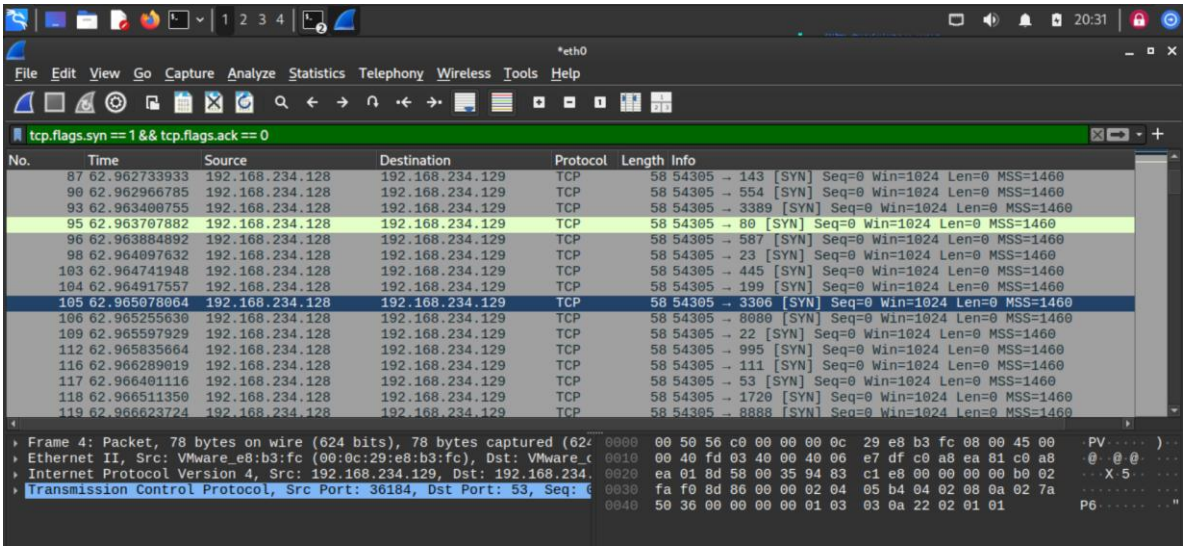
Para observar la huella del escaneo, se utilizó Wireshark con el filtro:

tcp.flags.syn == 1 && tcp.flags.ack == 0

La captura muestra múltiples segmentos TCP SYN enviados desde Kali (192.168.234.128) hacia Ubuntu (192.168.234.129) en diferentes puertos de destino. Este patrón corresponde al inicio de intentos de conexión utilizados para determinar qué puertos responden.

En la evidencia se observan paquetes dirigidos, entre otros, a los puertos 443, 3389, 445, 22, 80 y 21. La cantidad visible de paquetes es consistente con un escaneo de puertos; para la hoja de respuestas debe registrarse el número que aparece en el contador de paquetes de la captura original, si el docente exige una cifra exacta.

Figura 3. Tráfico de sondeo TCP SYN capturado durante el escaneo de puertos.



Captura de paquetes en Wireshark aplicando el filtro tcp.flags.syn == 1 && tcp.flags.ack == 0. Se evidencia la ráfaga de paquetes SYN generados por Nmap hacia diferentes puertos de VM2, patrón típico empleado por los analistas y detectable por sistemas IDS/IPS.

- **Respuestas de la práctica**

¿Qué es un ataque de reconocimiento?

Un ataque de reconocimiento es la fase en la que se recopila información sobre una red o sistema antes de realizar acciones más invasivas. Puede incluir la identificación de hosts

activos, direcciones IP, puertos abiertos, servicios, versiones de software y características del sistema operativo. Aunque algunas técnicas son pasivas, el escaneo activo genera tráfico que puede ser registrado por firewalls, IDS o sistemas de monitoreo.

¿Por qué Nmap es legal en una red propia, pero no en redes ajenas?

Nmap es una herramienta de doble uso. En una red propia o con autorización expresa permite realizar inventarios, comprobar configuraciones y evaluar la superficie de ataque. El mismo escaneo realizado contra una red ajena, sin permiso del propietario, puede considerarse una actividad no autorizada porque genera tráfico dirigido a sistemas de terceros y puede revelar información operativa. La legalidad depende de la autorización, el alcance definido y la legislación aplicable; por esa razón, el laboratorio se limitó a máquinas virtuales bajo control del estudiante.

2.2 Práctica 2. Captura y análisis de tráfico ARP, DNS y TCP

La segunda práctica se enfocó en observar protocolos de comunicación que suelen operar con mecanismos limitados de autenticación. Se analizaron ARP, DNS y TCP mediante Wireshark, además de utilizar arping, dig, ping y curl.

- **Solicitud y respuesta ARP**

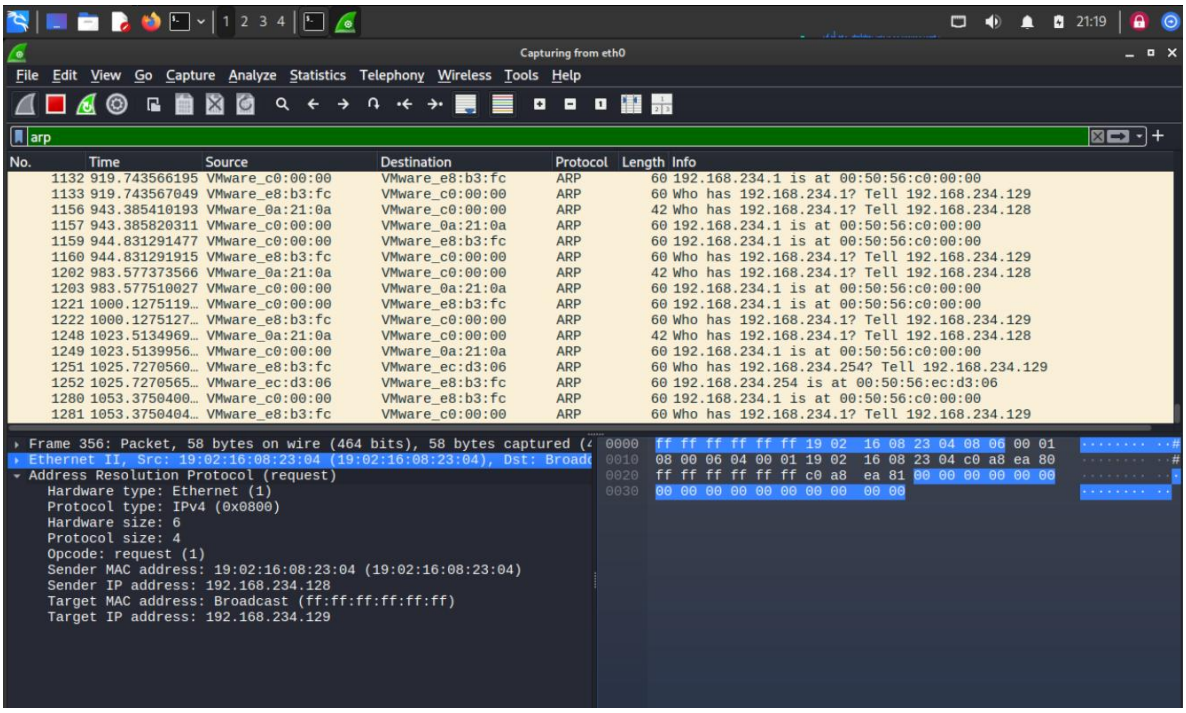
Para provocar una nueva resolución de dirección física se limpió la tabla de vecinos de Kali y se generó tráfico mediante un ping hacia Ubuntu. Wireshark se configuró con el filtro: *arp*.

La captura evidencia solicitudes ARP en broadcast. Kali pregunta quién posee la IP de Ubuntu y Ubuntu responde anunciando su dirección MAC. En la captura se observa la relación entre:

IP de Ubuntu: 192.168.234.129 MAC de Ubuntu: 00:0c:29:e8:b3:fc

El funcionamiento observado es sencillo: ARP permite descubrir qué dirección MAC corresponde a una dirección IPv4 dentro del segmento local. El problema es que el protocolo no incorpora autenticación fuerte del emisor.

Figura 4. Solicitud ARP capturada durante la resolución de la dirección de Ubuntu.



Captura de tráfico ARP en Wireshark durante la comunicación entre Kali Linux y Ubuntu. El paquete seleccionado corresponde a una solicitud ARP generada desde 192.168.234.128, que pregunta por la dirección MAC asociada con 192.168.234.129. El destino de la solicitud es la dirección de broadcast ff:ff:ff:ff:ff:ff, característica del proceso inicial de resolución ARP.

- **ARP gratuito y tabla de vecinos**

Se ejecutó el comando:

sudo arping -U -I eth0 -s 192.168.234.129 192.168.234.129 -c 3

El resultado indicó:

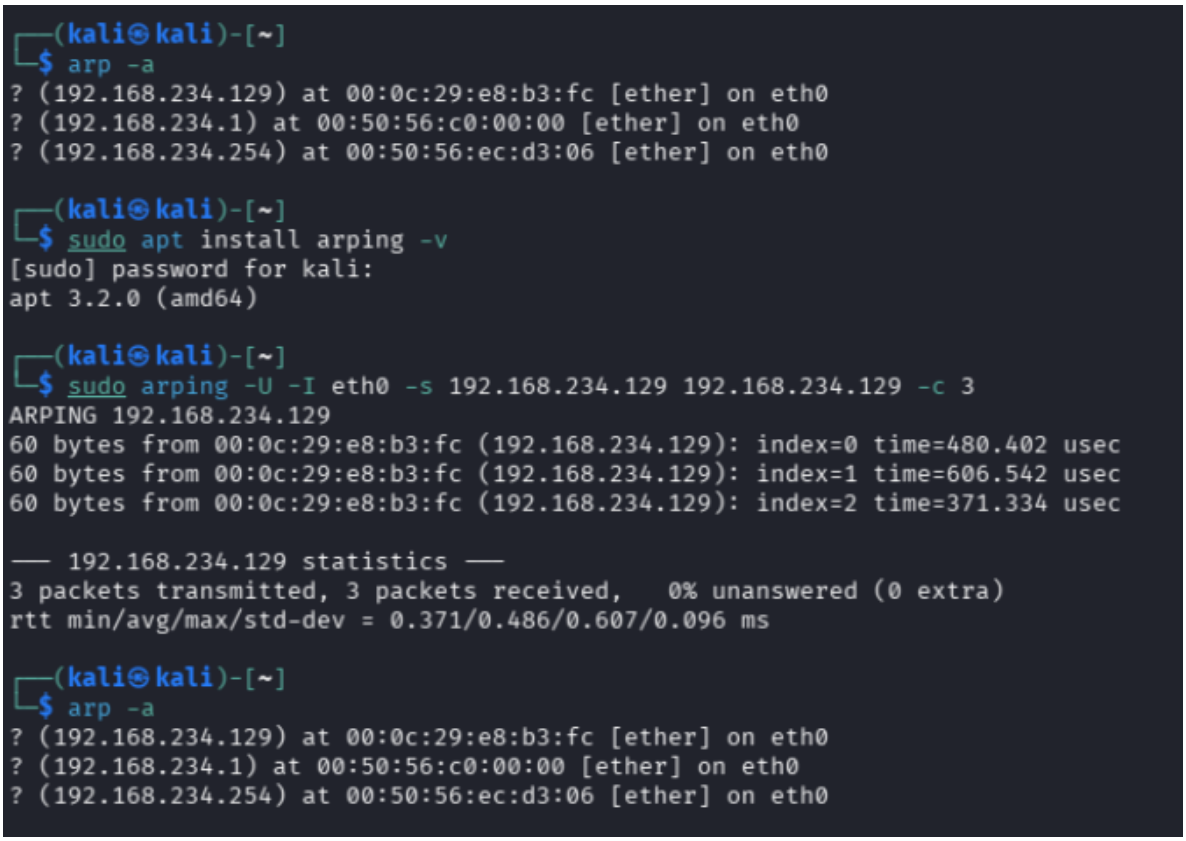
3 packets transmitted, 3 packets received 0 unanswered

Antes y después de la ejecución, la tabla ARP de Kali mostró la asociación:

192.168.234.129 → 00:0c:29:e8:b3:fc

La tabla local no presentó un cambio relevante. Esto es válido para la evidencia: el envío de un anuncio ARP no obliga a que el propio equipo emisor modifique su tabla. El riesgo del envenenamiento aparece cuando otro dispositivo acepta una asociación falsa y actualiza su caché con ella.

Figura 5. Envío de ARP Gratuito y verificación de la tabla ARP.



Ejecución de arping -U simulando un ARP Gratuito hacia la IP de VM2. Se compara la tabla ARP antes y después del comando para verificar el mecanismo que emplea el ARP Spoofing/Poisoning.

- **Análisis de tráfico DNS**

Se utilizó el filtro: dns

Después se generó una consulta con: dig google.com

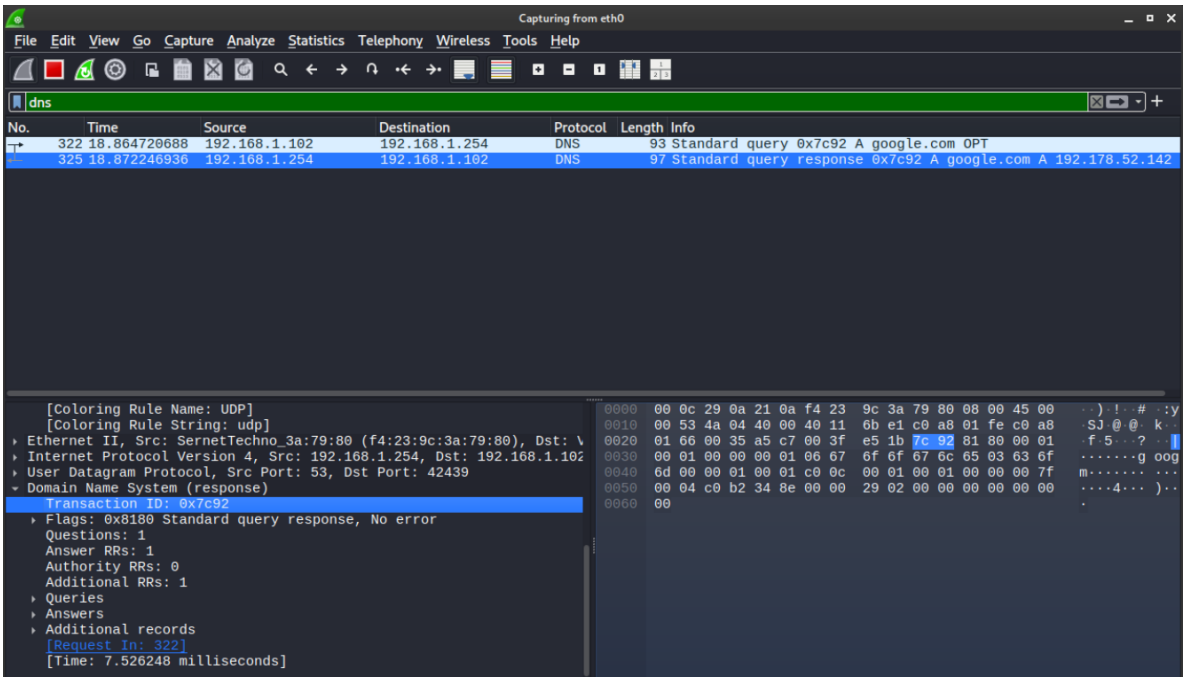
La captura muestra una consulta desde Kali (192.168.1.102) hacia el servidor DNS local (192.168.1.254) y la respuesta correspondiente. Debido al cambio posterior al modo puente, esta evidencia utiliza el segmento 192.168.1.0/24.

Los datos principales del protocolo son:

- **Puerto habitual de DNS:** UDP 53.
- **Puerto alternativo:** TCP 53, utilizado en determinados casos, por ejemplo, respuestas grandes o transferencia de zonas.
- **Dominio consultado:** google.com.
- **Tipo de registro:** A.
- **Transaction ID:** debe copiarse exactamente del campo mostrado en la captura de Wireshark.
- **Dirección devuelta:** debe copiarse del registro A visible en la respuesta.

La Transaction ID permite que el cliente relacione una respuesta DNS con la consulta que originó. Sin embargo, por sí sola no constituye autenticación del servidor; un atacante que consiga insertar o modificar respuestas podría intentar redirigir la resolución.

Figura 6. Análisis de una consulta y respuesta DNS mediante Wireshark.



Captura de una consulta DNS generada desde Kali Linux para resolver google.com. Se observa el intercambio entre Kali (192.168.1.102) y el servidor DNS (192.168.1.254), así como el Transaction ID, el puerto UDP 53 y el registro tipo A devuelto por el servidor.

- **Handshake TCP de tres vías**

Para establecer una conexión HTTP con Ubuntu se utilizó:

curl http://192.168.1.101

El filtro de Wireshark fue:

tcp && ip.addr == 192.168.1.101 && tcp.port == 80

La captura muestra el establecimiento de la conexión entre Kali (192.168.1.102) y Ubuntu (192.168.1.101):

1. Kali envía un paquete SYN.
2. Ubuntu responde con SYN, ACK.
3. Kali confirma con ACK.

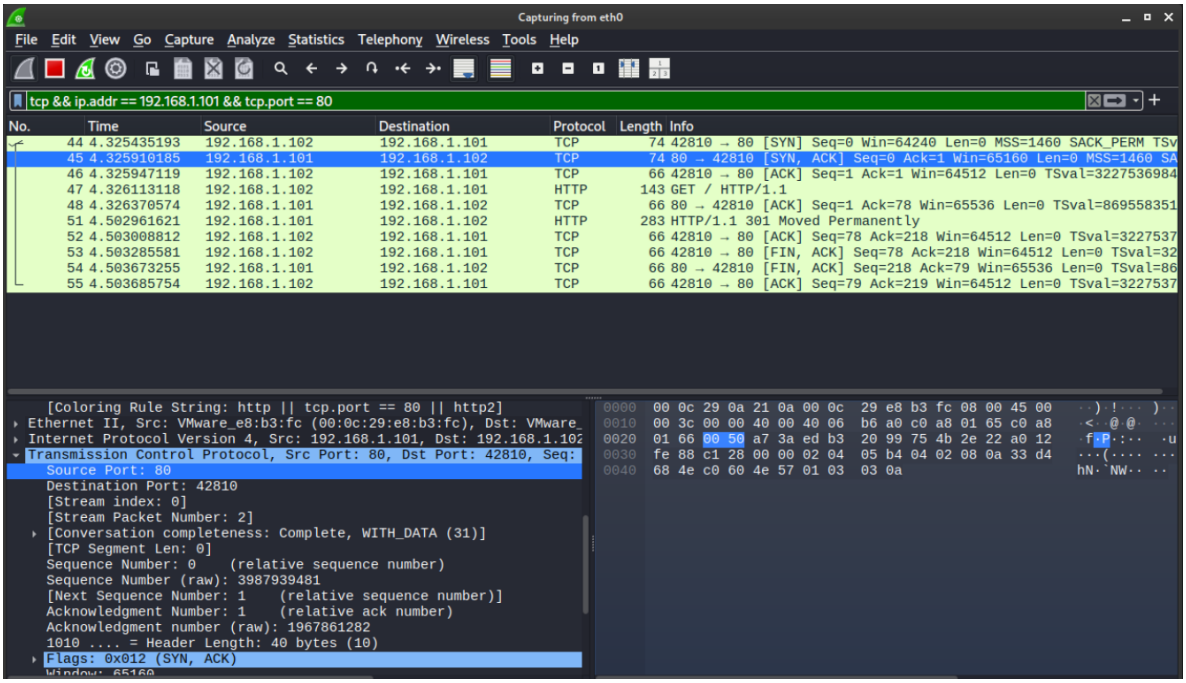
Los valores relativos observados fueron:

Paquete	Dirección	Flags	Secuencia	ACK
44	Kali ? Ubuntu	SYN	0	—
45	Ubuntu ? Kali	SYN, ACK	0	1
46	Kali ? Ubuntu	ACK	1	1

En el paquete seleccionado también aparecen los valores absolutos o *raw* de la captura:

- Número de secuencia: 3987939481
- Número de acuse: 1967861282

Figura 7. Establecimiento de conexión TCP mediante saludo de tres vías (Three-Way Handshake) entre Kali y Ubuntu.



Captura de paquetes en Wireshark mostrando el inicio de sesión TCP al puerto 80 (HTTP) entre Kali Linux (192.168.1.102) y Ubuntu (192.168.1.101). Se aprecian secuencialmente los paquetes SYN (No. 44), SYN-ACK (No. 45) y ACK (No. 46), así como los números de secuencia y acuse en la cabecera decodificada.

- **Respuestas de la práctica**

¿Por qué ARP es tan vulnerable?

ARP es vulnerable porque no fue diseñado con un mecanismo sólido de autenticación. Los equipos aceptan anuncios y respuestas ARP sin verificar de manera confiable si el emisor es realmente el propietario de la dirección IP anunciada. Además, las asociaciones IP-MAC se almacenan dinámicamente en cachés que pueden actualizarse con nueva información recibida dentro del segmento local.

¿Qué mecanismo hace posible el envenenamiento de caché ARP?

El envenenamiento es posible por la aceptación de respuestas ARP no solicitadas o gratuitas y por la actualización automática de la tabla de vecinos. Un dispositivo malicioso puede anunciar que su propia MAC corresponde a la IP de otro host, como el gateway o el servidor víctima. Si los demás equipos aceptan esa información, el atacante puede colocarse entre el origen y el destino, realizando un ataque de intermediario (*Man-in-the-Middle*).

¿En qué paso falla un ataque TCP SYN Flood?

El SYN Flood explota el momento posterior al primer paquete SYN. El servidor responde con SYN-ACK y reserva recursos mientras espera el tercer paquete, el ACK del cliente. Si ese ACK nunca llega, las conexiones quedan incompletas y pueden consumir la cola de conexiones pendientes del servidor.

2.3 Práctica 3. Criptografía en acción: hash, AES, RSA y TLS

La tercera práctica permitió comprobar de manera directa la diferencia entre integridad, confidencialidad, autenticación y protección del tráfico en tránsito. Se utilizaron comandos de Linux y OpenSSL para comparar archivos, cifrar mensajes y observar sesiones HTTP y HTTPS en Wireshark.

- **Funciones hash y efecto avalancha**

Se generaron dos archivos con una diferencia mínima:

Hola Mundo - ENSA Modulo 3

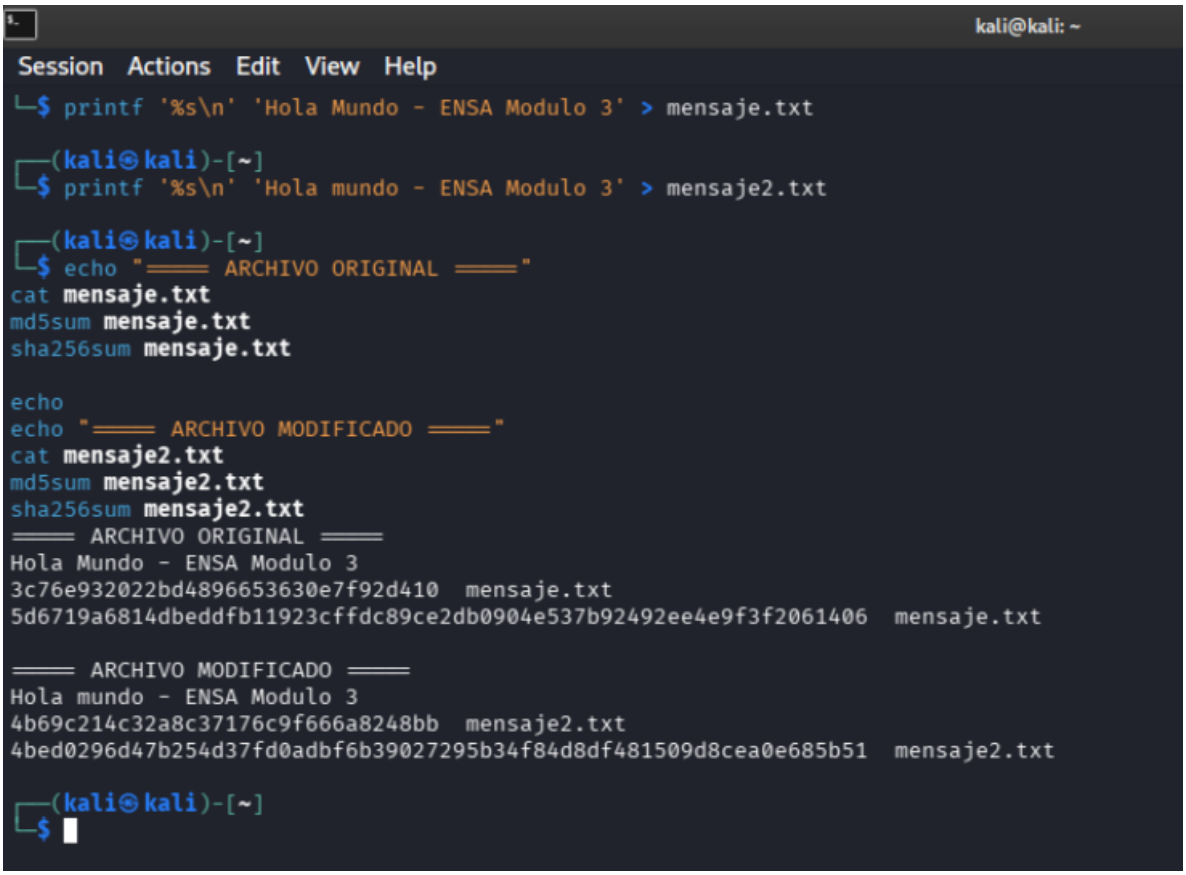
Hola mundo - ENSA Modulo 3

El único cambio fue la letra M mayúscula por m minúscula. Los hashes obtenidos fueron:

Archivo	MD5	SHA-256
mensaje.txt	3c76e932022bd4896653630e7f92d410	`5d6719a6814dbeddbf11923cffdc89e2db0904e537b92492ee49ef3f2061406
mensaje2.txt	4b69c214c32a8c37176c9f666a8248bb	4bed0296d47b254d37fd0adbf6b39027295b34f848d8d9dceae0e685b51

En el valor SHA-256 de mensaje2.txt debe conservarse exactamente la cadena que aparece en la captura original, sin introducir espacios. La evidencia confirma el efecto avalancha: una modificación muy pequeña produce una salida hash completamente diferente.

Figura 8. Comparación de hashes MD5 y SHA-256 antes y después de modificar un archivo.



Cálculo de los hashes MD5 y SHA-256 para un archivo original y una copia modificada en una sola letra. Aunque el cambio es mínimo, ambos valores hash cambian completamente, demostrando el efecto avalancha de las funciones hash y su utilidad para verificar la integridad de la información.

Como verificación complementaria se ejecutó:

sha256sum -c original.txt.sha256

El resultado original.txt: OK confirmó que el archivo coincidía con el valor SHA-256 registrado.

Figura 9. Verificación de integridad de un archivo mediante SHA-256.



Se genera un archivo de comprobación .sha256 y posteriormente se valida el archivo original mediante sha256sum -c. El resultado original.txt: OK confirma que el contenido coincide con el hash registrado y no ha sido modificado.

- **Cifrado simétrico AES-256**

Se creó el archivo secreto.txt con información confidencial y se cifró mediante AES-256-CBC, utilizando una derivación de clave con PBKDF2:

**openssl enc -aes-256-cbc -pbkdf2 **

**-in secreto.txt **

**-out secreto.enc **

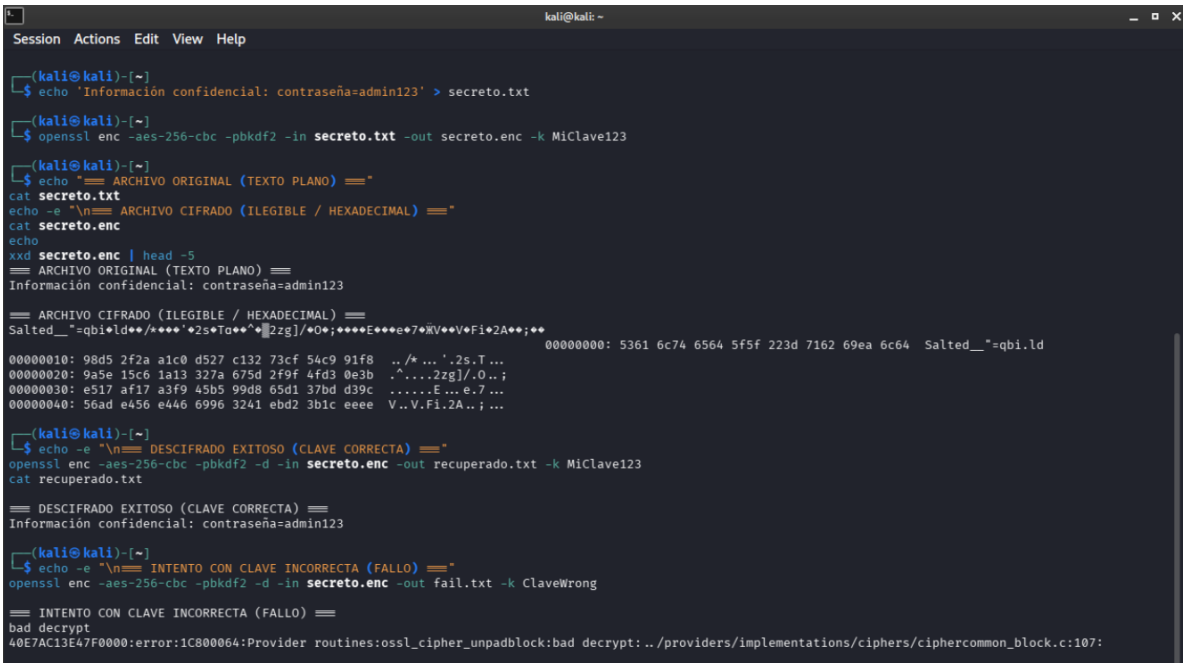
-k MiClave123

El contenido original podía leerse directamente:

Información confidencial: contraseña=admin123

Después del cifrado, el archivo secreto.enc dejó de ser legible como texto. La representación hexadecimal permitió observar que el contenido se había convertido en datos binarios. Al utilizar la contraseña correcta, el archivo se recuperó satisfactoriamente. En cambio, el intento con ClaveWrong produjo el error: **bad decrypt.**

Figura 10. Cifrado y descifrado simétrico de datos mediante AES-256-CBC.



Demostración del cifrado simétrico mediante OpenSSL utilizando AES-256-CBC. Se observa el contenido original en texto plano, el archivo cifrado representado en formato hexadecimal, la recuperación correcta del mensaje con la clave válida y el error producido al intentar descifrarlo con una clave incorrecta.

¿El cifrado AES cambia significativamente el tamaño del archivo?

El cifrado puede aumentar ligeramente el tamaño del archivo debido al vector de inicialización, el relleno por bloques y la información adicional utilizada por el formato de OpenSSL. Para archivos pequeños, ese aumento puede parecer proporcionalmente grande, aunque para archivos grandes suele ser pequeño en relación con el tamaño total.

- **Cifrado asimétrico RSA y HMAC**

Se generó un par de claves RSA:

- **privada.pem**
- **publica.pem**

La clave pública se utilizó para cifrar el mensaje:

Mensaje secreto RSA

Posteriormente, la clave privada permitió recuperar correctamente el contenido. El archivo cifrado msg.enc presentó un tamaño de 256 bytes, consistente con una clave RSA de 2048 bits para este ejercicio.

También se calcularon valores HMAC-SHA-256 para dos mensajes:

Mensaje original: Transferir 100 USD a Alex

Mensaje modificado: Transferir 10000 USD a Hacker

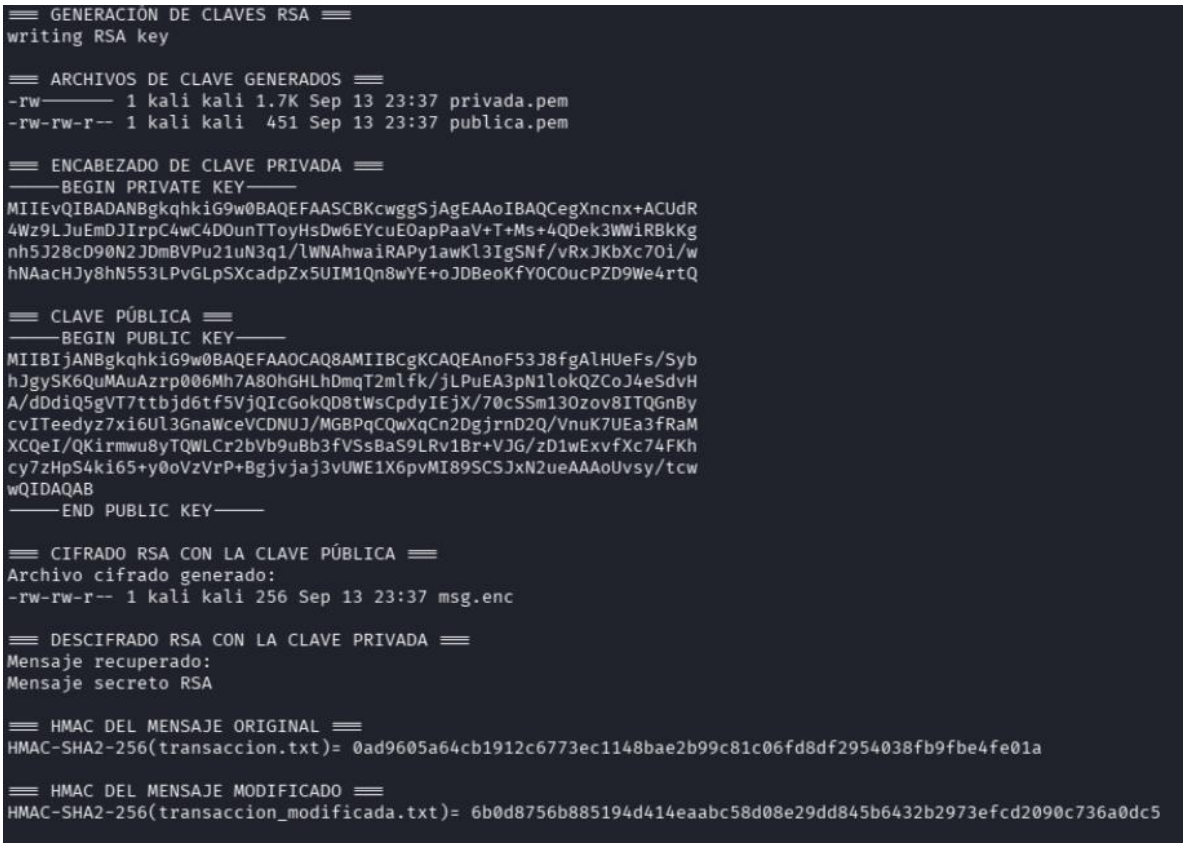
Los valores obtenidos fueron diferentes:

HMAC original: comienza con 0ad9605a64cb...

HMAC modificado: comienza con 6b0d8756b885...

La modificación del contenido provoca un HMAC diferente, por lo que el receptor puede detectar que el mensaje no conserva exactamente el contenido esperado, siempre que comparta la clave secreta correspondiente.

Figura 11. Generación de claves RSA, cifrado asimétrico, descifrado y verificación HMAC.



Generación de un par de claves RSA de 2048 bits y utilización de la clave pública para cifrar un mensaje. Posteriormente, la clave privada permite recuperar el contenido original. También se muestran dos valores HMAC diferentes, producidos a partir de un mensaje original y de una versión modificada, lo que permite detectar alteraciones en el contenido.

¿Cuál es la diferencia entre hash, cifrado y HMAC?

El hash genera una huella de longitud fija y no está diseñado para recuperar el mensaje original. El cifrado transforma la información para ocultarla y permite recuperarla mediante una clave. El HMAC combina una función hash con una clave secreta compartida para verificar tanto la integridad del mensaje como la autenticidad de quien conoce dicha clave.

- **Comparación entre HTTP y HTTPS**

Para observar tráfico HTTP local se ejecutó una petición hacia Apache en Ubuntu:

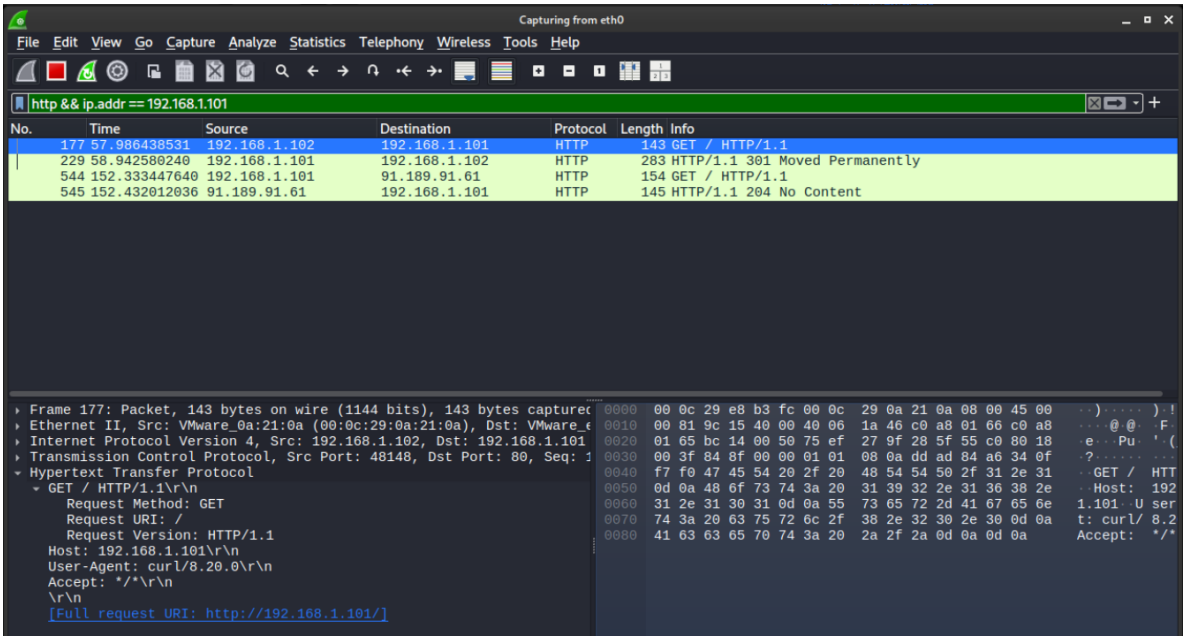
curl -v http://192.168.1.101/

La captura muestra:

- Kali: 192.168.1.102
- Ubuntu: 192.168.1.101
- Puerto destino: 80
- Método: GET
- URI: /
- Host: 192.168.1.101
- User-Agent: curl/8.20.0

Los encabezados HTTP se observan directamente en el panel de Wireshark. Esto demuestra que HTTP transmite la solicitud en texto claro.

Figura 12. Visualización de tráfico HTTP en texto claro.



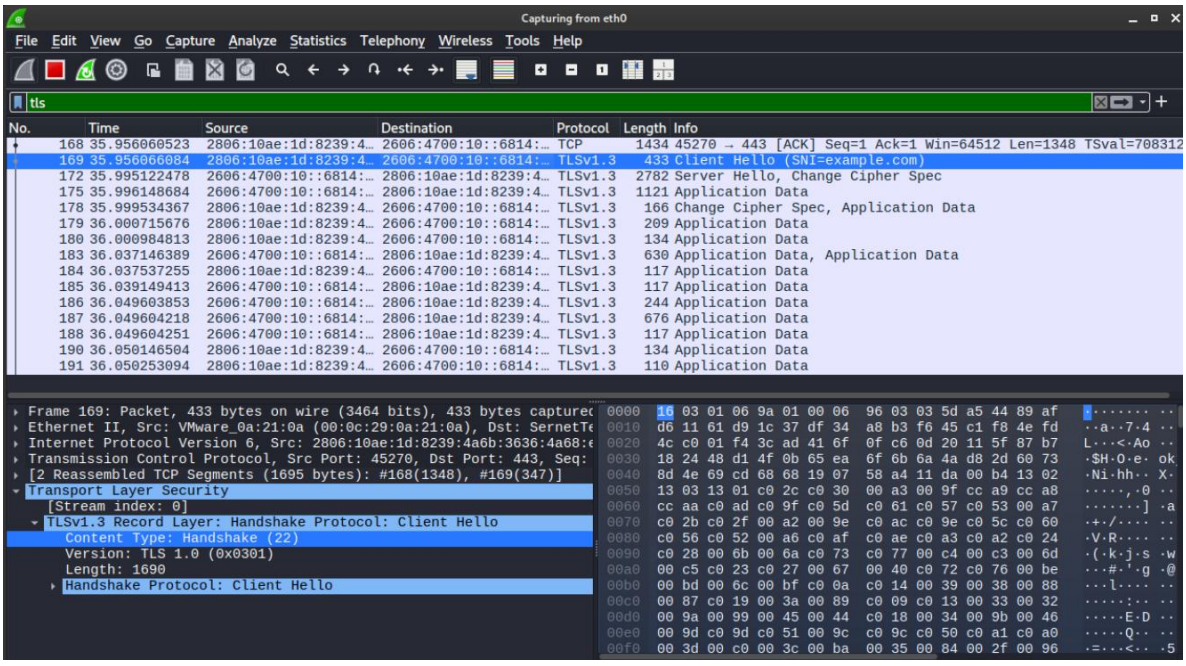
Captura de una comunicación HTTP entre Kali Linux (192.168.1.102) y Ubuntu (192.168.1.101). Al no utilizar cifrado TLS, Wireshark puede interpretar los encabezados y parte del contenido transmitido, lo que evidencia el riesgo de enviar información sensible mediante HTTP.

Para observar HTTPS se realizó una conexión hacia example.com y se aplicó el filtro: **tls**

La captura muestra el paquete número 169, identificado como: **TLSv1.3 Client Hello**

También se observan registros posteriores de tipo Application Data, cuyo contenido no se presenta directamente como texto HTTP.

Figura 13. Captura de negociación TLS y datos HTTPS cifrados.



Captura de una conexión HTTPS generada desde Kali Linux. Wireshark permite observar el proceso de negociación TLS, incluido el mensaje Client Hello, pero el contenido de la aplicación aparece como datos cifrados y no puede leerse directamente como texto HTTP.

- **Respuestas de la práctica**

¿Se puede leer el contenido del tráfico HTTP en Wireshark?

Sí. En la captura HTTP se observan el método GET, la URI, el host, el User-Agent y otros encabezados en texto claro.

¿Se puede leer directamente el contenido del tráfico HTTPS/TLS?

No. Wireshark muestra información de control y metadatos del protocolo, pero el contenido de aplicación aparece encapsulado como datos cifrados. Sin las claves de sesión correspondientes, no se puede interpretar directamente como una respuesta HTTP.

¿Qué versión de TLS se negoció?

La captura identifica la conexión como: TLSv1.3

Debe aclararse que dentro del detalle del registro pueden aparecer campos heredados de la estructura de TLS, como Version: TLS 1.0 (0x0301). Esa información pertenece al formato de compatibilidad del registro; la columna de protocolo y la negociación observada identifican la sesión como TLS 1.3.

¿En qué paquete se observa el Client Hello?

El mensaje Client Hello se observa en el paquete número: 169

3. Respuestas de estudiante

Hoja de Respuestas del Estudiante

Nombre: Gabriel René Díaz Lara
Fecha de la sesión: 13 de septiembre de 2026
Hora inicio: 20:15 hrs. | **Hora fin:** 23:45 hrs.
IP de VM1 (Kali): 192.168.234.128 (fase inicial) / 192.168.1.102 (fase final)
IP de VM2 (Víctima): 192.168.234.129 (fase inicial) / 192.168.1.101 (fase final)

Práctica 1 — Respuestas

¿Cuántos hosts encontró nmap -sn en tu subred?

4 hosts activos (192.168.234.1, 192.168.234.128, 192.168.234.129 y 192.168.234.254).

Lista los puertos abiertos encontrados en VM2:

Puerto: 21/tcp | Servicio: ftp (vsftpd 3.0.5)

Puerto: 22/tcp | Servicio: ssh (OpenSSH 10.0p2 Ubuntu 5ubuntu5.1)

Puerto: 53/tcp | Servicio: domain (ISC BIND 9.20.11-1ubuntu2.2)

Puerto: 80/tcp | Servicio: http (Apache httpd 2.4.64)

Puerto: 443/tcp | Servicio: ssl/http (Apache httpd 2.4.64)

¿Cuántos paquetes SYN sin respuesta contaste en Wireshark?
995 paquetes (coincidente con los 995 closed tcp ports (reset) reportados por Nmap durante el barrido sobre los 1000 puertos por defecto).

Reflexión: ¿Qué es un ataque de reconocimiento y por qué nmap es legal en tu propia red?
El reconocimiento constituye la etapa inicial del ciclo de intrusión donde se analiza la topología y se mapean servicios, versiones de demonios y posibles vías de entrada sin explotar fallos de forma directa. Nmap opera como un instrumento técnico neutro de auditoría: en infraestructura propia o bajo contrato expreso de evaluación es un mecanismo legítimo de verificación defensiva; sin embargo, proyectar ese mismo tráfico de sondeo sobre activos de terceros sin autorización previa quebranta el principio de consentimiento informado y vulnera las normas de acceso no autorizado tipificadas en el marco jurídico penal.

Práctica 2 — Respuestas

MAC de VM2 según arp -a:

00:0c:29:e8:b3:fc

¿Qué sucedió con la tabla ARP después del arping -U?

La tabla local en Kali no sufrió alteración alguna y conservó la correspondencia legítima 192.168.234.129 at 00:0c:29:e8:b3:fc. La emisión de tramas ARP gratuitas (-U) tiene un propósito de difusión hacia el segmento para obligar a que *otros* nodos del enlace actualicen sus tablas de vecinos, sin forzar una sobreescritura interna en la pila del propio equipo emisor.

Transaction ID de la primera consulta DNS:

0x7c92

IP devuelta para google.com:

192.178.52.142 (con respuesta resuelta vía el DNS local en 192.168.1.254).

Número de secuencia del SYN (paquete 1 del handshake TCP):

Secuencia relativa: 0 (Número de secuencia absoluto / raw: 3987939481, en el paquete No. 44).

Reflexión: ¿Por qué ARP es tan vulnerable al spoofing?

El protocolo carece de mecanismos criptográficos de firma o autenticación de origen. Los sistemas operativos aceptan y almacenan de forma pasiva respuestas ARP incluso cuando no han emitido una solicitud previa (respuestas gratuitas o no solicitadas), actualizando su memoria caché bajo un modelo de confianza ciega en el medio físico compartido. Esta debilidad estructural permite a cualquier host malicioso inyectar tramas falsas y atribuirse la dirección IP del router o de un servidor objetivo, consumando un ataque de intermediario (*Man-in-the-Middle*) a nivel de enlace de datos.

Práctica 3 — Respuestas

Hash MD5 de mensaje.txt:

3c76e932022bd4896653630e7f92d410

Hash MD5 de mensaje2.txt (con 'm' minúscula):

4b69c214c32a8c37176c9f666a8248bb

¿El cifrado AES cambia significativamente el tamaño del archivo?

No de manera desproporcionada. Se produce únicamente un incremento mínimo derivado del encabezado del formato (Salted__ de 16 bytes en la implementación de OpenSSL) y del mecanismo de relleno por bloques (*PKCS#7 padding*) para completar el múltiplo de 16 bytes que exige el algoritmo AES-256 en modo CBC.

¿Puedes leer el tráfico HTTP en Wireshark?

SÍ. La solicitud viaja en texto claro, permitiendo visualizar directamente el método GET / HTTP/1.1, encabezados de usuario (curl/8.20.0), host y parámetros de red.
¿Y el HTTPS/TLS?

NO. Toda la carga útil se encuentra protegida bajo registros criptográficos de tipo *Application Data*, impidiendo reconstruir la solicitud web o los datos transmitidos sin acceso a las claves simétricas de la sesión.

Versión de TLS negociada en la conexión HTTPS:

TLSv1.3 (visible en el protocolo del registro y en la negociación posterior al intercambio de tramas iniciales).

Reflexión: Diferencia entre cifrado simétrico y asimétrico:

El cifrado simétrico (como AES) emplea una clave única y secreta tanto para cifrar como para descifrar, destacando por su alta eficiencia computacional y velocidad en el tratamiento de grandes volúmenes de datos en reposo o flujos continuos. Por su parte, el cifrado asimétrico (como RSA) utiliza un par matemático diferenciado compuesto por una clave pública para cifrar y una clave privada, celosamente resguardada, para descifrar, solventando el problema de la distribución de secretos en canales inseguros. En la arquitectura de seguridad moderna se combinan mediante esquemas híbridos: la criptografía asimétrica autentica a los extremos e intercambia una clave de sesión efímera, mientras que el algoritmo simétrico asume el cifrado ágil de todo el tráfico subsecuente.

4. Conclusión

La ejecución de este laboratorio deja al descubierto una realidad técnica que suele minimizarse en el diseño de arquitecturas corporativas: la confianza ciega en las capas inferiores del modelo de interconexión es una negligencia operativa. Ver en Wireshark cómo un protocolo fundacional como ARP acepta de manera pasiva que cualquier host dentro del mismo dominio de difusión se atribuya la identidad de la puerta de enlace, o constatar que una petición HTTP exhibe credenciales y cabeceras en texto claro frente a un analizador pasivo, demuestra que el perímetro físico ya no contiene absolutamente nada. La información que transita sin mecanismos de cifrado robusto o sin validación de integridad no corre el riesgo de ser interceptada; está, de facto, entregada a cualquier actor que logre posicionarse en el segmento local.

El verdadero problema de fondo no radica únicamente en las carencias intrínsecas de diseño de protocolos concebidos en décadas pasadas (cuando las redes académicas asumían benevolencia mutua entre nodos), sino en la ausencia sistemática de gobernanza y políticas internas de seguridad dentro de las organizaciones. Implementar mecanismos como *Dynamic ARP Inspection* (DAI), *DHCP Snooping*, segmentación estricta mediante VLANs y la obligatoriedad de TLS 1.3 no requiere tecnología experimental ni presupuestos astronómicos, sino voluntad de configuración y cumplimiento normativo. Cuando una empresa prescinde de una política de seguridad basada en estándares reconocidos —como ISO/IEC 27001 o NIST SP 800-53— la infraestructura queda expuesta a un estado de indefensión técnica donde la detección de intrusiones depende de la suerte y no de la observabilidad estructurada.

Trasladando este escenario técnico al plano jurídico mexicano, si las maniobras de reconocimiento activo, intercepción de paquetes y suplantación ejecutadas en estas prácticas se hubiesen proyectado contra infraestructura ajena y sin consentimiento contractual, las consecuencias legales habrían sido severas e inmediatas.

El Código Penal Federal tipifica con meridiana claridad estas conductas en su Título Noveno, bajo el rubro de delitos contra la indemnidad y privacidad de la información. En específico, el **Artículo 211 bis 1** sanciona a quien, sin la debida autorización, modifique, destruya o provoque pérdida de información contenida en sistemas o equipos de informática protegidos por algún mecanismo de seguridad. El sondeo sistemático de puertos y la posterior inyección de tramas gratuitas de ARP para desviar el flujo de paquetes encuadran plenamente en la figura de acceso no autorizado y

alteración indebida de los canales de comunicación de un sistema. La gravedad de las penas privativas de la libertad se incrementa de forma sustantiva si tales acciones se ejercen contra sistemas del Estado o instituciones que integran el sistema financiero nacional (conforme a los subsecuentes artículos 211 bis 2 y 211 bis 4).

La intercepción de tráfico en texto claro —como la observada en la sesión HTTP no cifrada— activa de lleno las responsabilidades estipuladas en la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP). Dicho ordenamiento impone a los responsables del tratamiento la obligación ineludible de adoptar y mantener medidas de seguridad administrativas, técnicas y físicas que garanticen la confidencialidad de la información y eviten su daño, pérdida, alteración o acceso no autorizado (Artículos 19 y 21). Exponer identificadores de usuario, credenciales o transacciones por operar servicios web desprovistos de TLS constituye un incumplimiento flagrante de los principios de seguridad y confidencialidad tutelados por la ley. En caso de una brecha provocada por un intermediario (*Man-in-the-Middle*), la organización se enfrentaría a sanciones económicas millonarias impuestas por el INAI, al margen de las eventuales demandas civiles por daños y perjuicios derivadas de la filtración.

El aprendizaje de fondo trasciende el dominio de los comandos en Kali o la lectura de tramas hexadecimales en Wireshark. La seguridad en redes no puede asumirse como un parche superficial que se agrega al final del despliegue; debe concebirse como un principio de diseño indivisible donde los controles criptográficos, la vigilancia continua y el estricto apego al marco legal son las únicas barreras reales frente a la materialización de un delito informático.

5. Bibliografía

Cámara de Diputados del H. Congreso de la Unión. (2025). *Ley Federal de Protección de Datos Personales en Posesión de los Particulares* (última reforma publicada en el Diario Oficial de la Federación el 14 de noviembre de 2025). https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf

Código Penal Federal, Capítulo II, Título Noveno, Libro Segundo, artículos 211 bis 1 al 211 bis 7. (2025). *Acceso ilícito a sistemas y equipos de informática*. Justia México. https://mexico.justia.com/federales/codigos/codigo-penal-federal/libro-segundo/titulo-noveno/capitulo-ii/

Internet Engineering Task Force. (1982). *Internet protocol: DARPA Internet program protocol specification* (RFC 793). Referencia técnica consultada mediante el repositorio de RFC de Wireshark

Internet Engineering Task Force. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). RFC Editor. https://www.rfc-editor.org/info/rfc8446/

Nmap Project. (s. f.-a). *Nmap documentation*. https://nmap.org/docs.html

Nmap Project. (s. f.-b). *Nmap reference guide: Network exploration and security auditing*. https://nmap.org/book/man.html

OpenSSL Project. (s. f.). *openssl-enc: Symmetric cipher routines*. OpenSSL Documentation. https://docs.openssl.org/3.3/man1/openssl-enc/

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc8446/

Wireshark Foundation. (s. f.-a). *RFC*. Wireshark Wiki. https://wiki.wireshark.org/RFC

Wireshark Foundation. (s. f.-b). *Wireshark user's guide*. https://www.wireshark.org/docs/wsug_html/